

CO2 AT2 Programs

1. Data Preprocessing, Aggregation and Visualization

Aim:

To preprocess employee data, perform aggregation and visualize results.

Algorithm:

1. Import libraries
2. Create DataFrame
3. Clean data
4. Fill missing salary
5. Convert date
6. Calculate years of service
7. Aggregate department data
8. Display bar and scatter plots

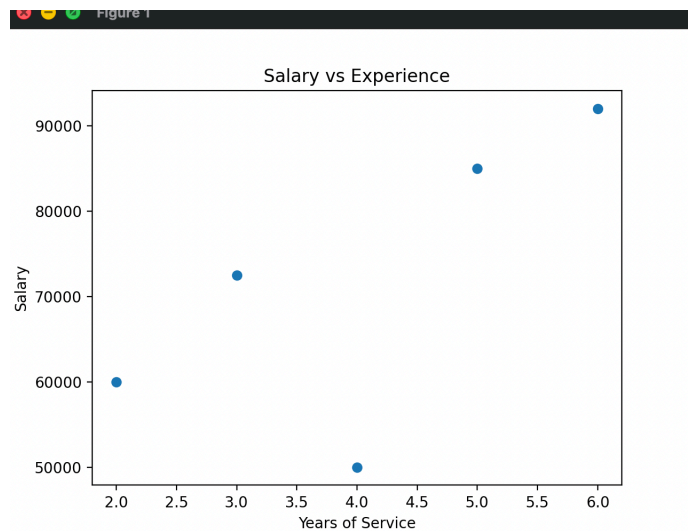
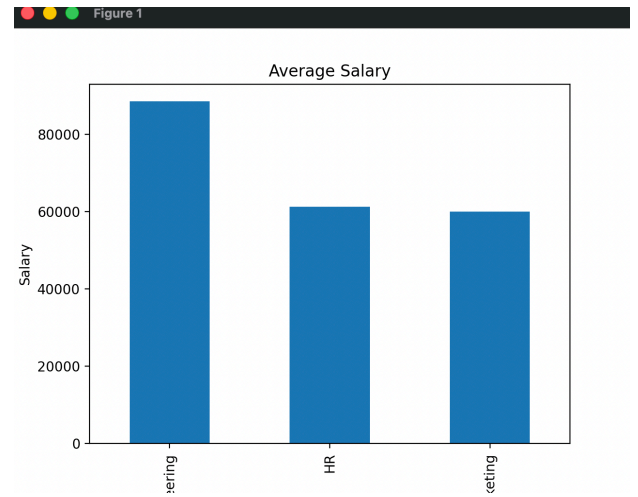
Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data={"Employee_Name":[" Alice ", "Bob", "Charlie", "Diana", "Evan"],
      "Department":["HR", "Engineering", "HR", "Engineering", "Marketing"],
      "Salary":[50000, 85000, np.nan, 92000, 60000],
      "Join_Date":["2022-01-15", "2021-06-20", "2023-03-11", "2020-11-01", "2024-02-28"]}
df=pd.DataFrame(data)
df["Employee_Name"]=df["Employee_Name"].str.strip()
df["Salary"]=df["Salary"].fillna(df["Salary"].median())
df["Join_Date"]=pd.to_datetime(df["Join_Date"])
df["Years_of_Service"]=2026-df["Join_Date"].dt.year
summary=df.groupby("Department").agg(Avg_Salary= ("Salary", "mean"), Total_Employe
es= ("Employee_Name", "count"))
print(df)
print(summary)
summary["Avg_Salary"].plot(kind="bar")
plt.show()
plt.scatter(df["Years_of_Service"], df["Salary"])
plt.show()
```

Output:

```
p69.py > ...
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Sample Data
6 data = {
7     "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
8     "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
9     "Salary": [50000, 85000, np.nan, 92000, 60000],
10    "Join_Date": ["2022-01-15", "2021-06-20", "2023-03-11", "2020-11-01", "2024-02-28"]
11 }
12
13 df = pd.DataFrame(data)
14
15 # Remove spaces
16 df["Employee_Name"] = df["Employee_Name"].str.strip()
17
18 # Fill missing salary
19 df["Salary"] = df["Salary"].fillna(df["Salary"].median())
20
21 # Convert date
22 df["Join_Date"] = pd.to_datetime(df["Join_Date"])
23
24 # Years of Service
25 df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year
26
27 print(df)
28
29 # Aggregate
30 summary = df.groupby("Department").agg(
31
32 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
33
34 saicharan@sais-MacBook-Air vscode_practice % python3 p69.py
35 Employee_Name Department Salary Join_Date Years_of_Service
36 0 Alice HR 50000.0 2022-01-15 4
37 1 Bob Engineering 85000.0 2021-06-20 5
38 2 Charlie HR 72500.0 2023-03-11 3
39 3 Diana Engineering 92000.0 2020-11-01 6
40 4 Evan Marketing 60000.0 2024-02-28 2
41
42 Department Avg_Salary Total_Employees
43 Engineering 88500.0 2
44 HR 61250.0 2
45 Marketing 60000.0 1
```



Result:

Employee data is successfully preprocessed, aggregated and visualized.

2. Create DataFrame

Aim:

To create a simple DataFrame.

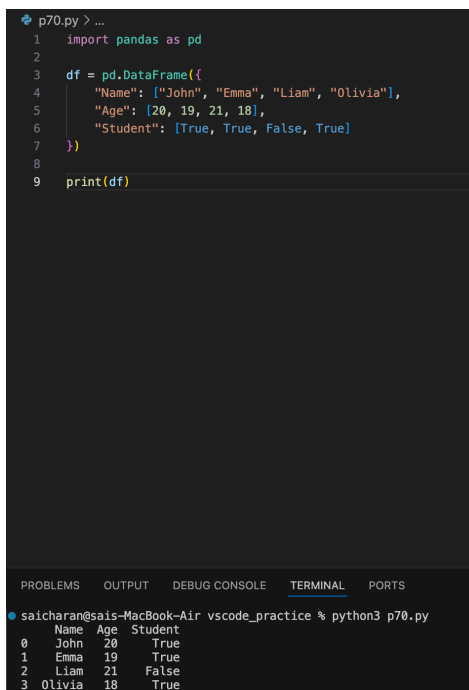
Algorithm:

1. Import pandas
2. Create DataFrame
3. Print DataFrame

Code:

```
import pandas as pd
df=pd.DataFrame({"Name":["John","Emma","Liam","Olivia"],"Age":
[20,19,21,18],"Student":[True,True,False,True]})
print(df)
```

Output:



The screenshot shows a VS Code editor with a Python file named p70.py. The code defines a pandas DataFrame with columns 'Name', 'Age', and 'Student'. The 'Name' column contains ['John', 'Emma', 'Liam', 'Olivia'], 'Age' contains [20, 19, 21, 18], and 'Student' contains [True, True, False, True]. The DataFrame is printed to the terminal.

```
p70.py > ...
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "Name": ["John", "Emma", "Liam", "Olivia"],
5     "Age": [20, 19, 21, 18],
6     "Student": [True, True, False, True]
7 })
8
9 print(df)
```

The terminal output shows the DataFrame as follows:

	Name	Age	Student
0	John	20	True
1	Emma	19	True
2	Liam	21	False
3	Olivia	18	True

Result:

DataFrame created successfully.

3. Add Row

Aim:

To add a new row to DataFrame.

Algorithm:

1. Create DataFrame
2. Create new row
3. Concatenate
4. Print

Code:

```
import pandas as pd
df=pd.DataFrame({"Name":["John","Emma","Liam","Olivia"],"Age":
[20,19,21,18],"Student":[True,True,False,True]})
new_row=pd.DataFrame([["Sophia",22,False]],columns=["Name","Age","Student"])
df=pd.concat([df,new_row],ignore_index=True)
print(df)
```

Output:



The screenshot shows a VS Code editor with a Python file named p71.py. The code defines a DataFrame with columns 'Name', 'Age', and 'Student'. It then creates a new row for 'Sophia' and appends it to the existing DataFrame. The output in the terminal shows the resulting DataFrame with 5 rows.

```
p71.py / ...
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "Name": ["John", "Emma", "Liam", "Olivia"],
5     "Age": [20, 19, 21, 18],
6     "Student": [True, True, False, True]
7 })
8
9 new_row = pd.DataFrame([["Sophia",22,False]],
10 | | | | | columns=["Name","Age","Student"])
11
12 df = pd.concat([df,new_row],ignore_index=True)
13
14 print(df)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● saicharan@sais-MacBook-Air vscode_practice % python3 p71.py
   Name  Age  Student
0  John   20     True
1  Emma   19     True
2  Liam   21    False
3  Olivia  18     True
4  Sophia  22    False
```

Result:

Row added successfully.

4. Mean Median Mode

Aim:

To find mean, median and mode.

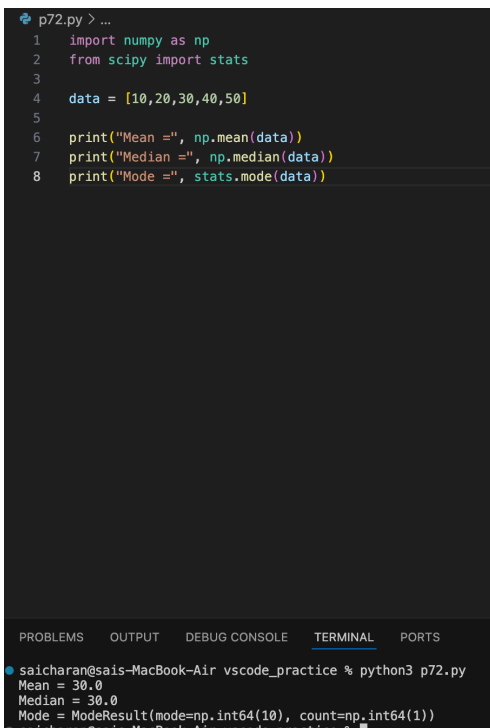
Algorithm:

1. Import libraries
2. Create list
3. Calculate statistics

Code:

```
import numpy as np
from scipy import stats
data=[10,20,30,40,50]
print(np.mean(data))
print(np.median(data))
print(stats.mode(data))
```

Output:



```
p72.py > ...
1  import numpy as np
2  from scipy import stats
3
4  data = [10,20,30,40,50]
5
6  print("Mean =", np.mean(data))
7  print("Median =", np.median(data))
8  print("Mode =", stats.mode(data))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@sais-MacBook-Air vscode_practice % python3 p72.py
Mean = 30.0
Median = 30.0
Mode = ModeResult(mode=np.int64(10), count=np.int64(1))
```

Result:

Statistics calculated successfully.

5. Probability Distribution

Aim:

To calculate cumulative probability.

Algorithm:

1. Import norm
2. Calculate cdf
3. Print

Code:

```
from scipy.stats import norm
print(norm.cdf(85, loc=70, scale=10))
```

Output:



The image shows a screenshot of a VS Code editor window. The editor has a dark theme. The top part shows a Python file named 'p73.py' with the following code:

```
1 from scipy.stats import norm
2
3 probability = norm.cdf(85, loc=70, scale=10)
4
5 print("Probability =", probability)
```

Below the editor, there is a terminal window. The terminal shows the command 'python3 p73.py' being executed, and the output is 'Probability = 0.9331927987311419'.

Result:

Probability calculated.

6. Hypothesis Testing

Aim:

To perform one sample t-test.

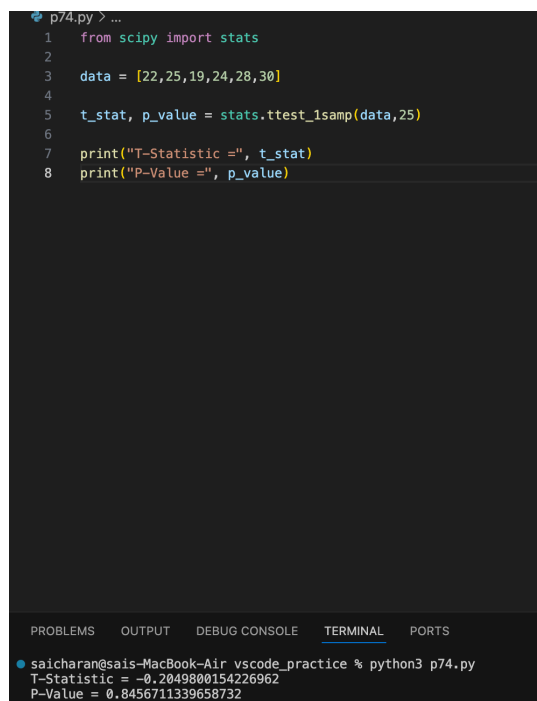
Algorithm:

1. Import stats
2. Create data
3. Perform t-test
4. Print

Code:

```
from scipy import stats
data=[22,25,19,24,28,30]
t,p=stats.ttest_1samp(data,25)
print(t,p)
```

Output:



The screenshot shows a VS Code editor with a file named 'p74.py' containing the following Python code:

```
1 from scipy import stats
2
3 data = [22,25,19,24,28,30]
4
5 t_stat, p_value = stats.ttest_1samp(data,25)
6
7 print("T-Statistic =", t_stat)
8 print("P-Value =", p_value)
```

Below the editor, the 'TERMINAL' tab is active, displaying the command and output:

```
saicharan@sais-MacBook-Air vscode_practice % python3 p74.py
T-Statistic = -0.2049800154226962
P-Value = 0.8456711339658732
```

Result:

Hypothesis test completed.

7. Correlation

Aim:

To find Pearson correlation.

Algorithm:

1. Import pearsonr
2. Create data
3. Find correlation

Code:

```
from scipy.stats import pearsonr
x=[1,2,3,4,5]
y=[2,4,6,8,10]
print(pearsonr(x,y))
```

Output:



The image shows a screenshot of a VS Code editor window. The editor has a dark theme. At the top, the file name is 'p75.py > ...'. The code in the editor is as follows:

```
1  from scipy.stats import pearsonr
2
3  x = [1,2,3,4,5]
4  y = [2,4,6,8,10]
5
6  corr, p = pearsonr(x,y)
7
8  print("Correlation =", corr)
```

Below the editor, there is a panel with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is selected. The terminal shows the command 'python3 p75.py' and the output 'Correlation = 1.0'.

Result:

Correlation calculated.

8. Linear Algebra

Aim:

To solve linear equations.

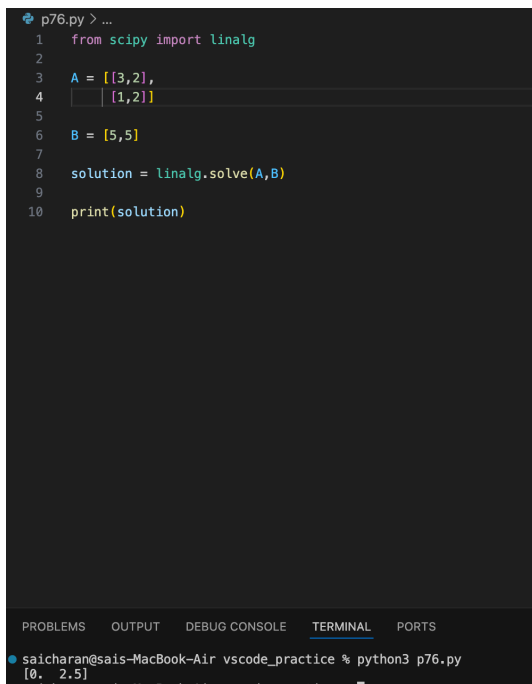
Algorithm:

1. Import linalg
2. Define matrices
3. Solve

Code:

```
from scipy import linalg
A=[[3,2],[1,2]]
B=[5,5]
print(linalg.solve(A,B))
```

Output:



The image shows a screenshot of a VS Code editor window. The editor has a dark theme. At the top, the file name 'p76.py' is visible. The code in the editor is as follows:

```
1 from scipy import linalg
2
3 A = [[3,2],
4      [1,2]]
5
6 B = [5,5]
7
8 solution = linalg.solve(A,B)
9
10 print(solution)
```

Below the editor, there is a terminal panel with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing the command 'python3 p76.py' and its output:

```
saicharan@sais-MacBook-Air vscode_practice % python3 p76.py
[0.  2.5]
```

Result:

Linear equations solved.

9. Optimization

Aim:

To minimize a function.

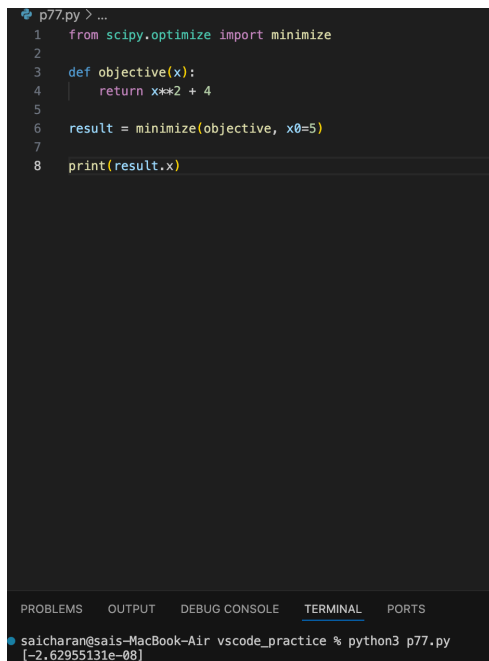
Algorithm:

1. Import minimize
2. Define function
3. Optimize

Code:

```
from scipy.optimize import minimize
def objective(x):
    return x**2+4
print(minimize(objective,x0=5).x)
```

Output:



The image shows a screenshot of a VS Code editor window. The editor is open to a file named 'p77.py'. The code in the file is as follows:

```
1 from scipy.optimize import minimize
2
3 def objective(x):
4     return x**2 + 4
5
6 result = minimize(objective, x0=5)
7
8 print(result.x)
```

Below the editor, the 'TERMINAL' panel is visible, showing the output of running the script:

```
saicharan@saais-MacBook-Air vscode_practice % python3 p77.py
[-2.62955131e-08]
```

Result:

Optimization completed.

10. Logistic Regression

Aim:

To classify Iris dataset.

Algorithm:

1. Load dataset
2. Split data
3. Scale
4. Train
5. Predict
6. Accuracy

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

iris=load_iris()
X_train,X_test,y_train,y_test=train_test_split(iris.data,iris.target,test_size=
0.3,random_state=42)
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(X_test)
model=LogisticRegression()
model.fit(X_train,y_train)
print(accuracy_score(y_test,model.predict(X_test)))
```

Output:



```
p78.py > ...
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import accuracy_score
6
7 iris = load_iris()
8
9 X = iris.data
10 y = iris.target
11
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.3, random_state=42
14 )
15
16 scaler = StandardScaler()
17
18 X_train = scaler.fit_transform(X_train)
19 X_test = scaler.transform(X_test)
20
21 model = LogisticRegression()
22
23 model.fit(X_train, y_train)
24
25 y_pred = model.predict(X_test)
26
27 print("Accuracy =", accuracy_score(y_test, y_pred))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
saicharan@sais-MacBook-Air vscode_practice % python3 p78.py
Accuracy = 1.0
saicharan@sais-MacBook-Air vscode_practice %
```

Result:

Model classified successfully.

11. KMeans

Aim:

To cluster Iris dataset.

Algorithm:

1. Load dataset
2. Create KMeans
3. Fit model
4. Print labels

Code:

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
iris=load_iris()
k=KMeans(n_clusters=3,random_state=42)
k.fit(iris.data)
print(k.labels_)
```

Output:

```

1 from sklearn.datasets import load_iris
2 from sklearn.cluster import KMeans
3
4 iris = load_iris()
5
6 kmeans = KMeans(n_clusters=3, random_state=42)
7
8 kmeans.fit(iris.data)
9
10 print("Cluster Labels:")
11 print(kmeans.labels_)

```

Result:

Data clustered successfully.

